

Gérer les attributs class et style : th:class, th:classappend, th:style, th:styleappend

1. Pourquoi ces attributs ?

Très souvent, il faut donner une valeur à l'attribut class ou style d'un élément en fonction d'une condition : une ligne de tableau colorée selon un statut, un bouton désactivé, un champ marqué en erreur, etc. Thymeleaf propose quatre attributs pour cela :

Attribut	Cible	Comportement
th:class	class	Remplace la valeur de class (la crée si elle n'existe pas)
th:classappend	class	Ajoute une valeur à la fin de class
th:style	style	Remplace la valeur de style
th:styleappend	style	Ajoute une valeur à la fin de style

Pour les exemples, le contrôleur place un booléen dans le modèle :

```
@GetMapping("/class-example")
public String classExample(Model model) {
    model.addAttribute("isAdmin", true); // essayer aussi avec false
    return "class-example";
}
```

2. th:class

th:class crée l'attribut class lors de la génération du HTML, ou **remplace** celui qui existe déjà.

```
<!-- Template -->
<p class="user-class" th:class="${isAdmin} ? 'admin-class' : 'user-class'">
    Texte 1
</p>

<p th:class="${isAdmin} ? 'admin-class' : 'user-class'">
    Texte 2
</p>

<p th:class="${isAdmin} ? 'admin-class' : ''">
    Texte 3
</p>
```

Si isAdmin est vrai, on obtient :

```
<p class="admin-class">Texte 1</p>
<p class="admin-class">Texte 2</p>
<p class="admin-class">Texte 3</p>
```

Si isAdmin est faux :

```
<p class="user-class">Texte 1</p>
<p class="user-class">Texte 2</p>
<p>Texte 3</p>
```

Remarques importantes :

- Les valeurs sont écrites entre apostrophes ('admin-class'), car ce sont des littéraux de chaîne. Thymeleaf tolère certains littéraux sans apostrophes (lettres, chiffres, tirets, soulignés), mais les apostrophes évitent les ambiguïtés et les erreurs de frappe.
- Dans le premier paragraphe, le class="user-class" écrit à côté de th:class ne sert qu'à l'affichage de la maquette dans un navigateur (*Natural Template*) : il est remplacé au traitement.
- **Si th:class produit une valeur vide, l'attribut class est supprimé en entier**, y compris une valeur statique écrite à côté. C'est le cas du troisième paragraphe quand isAdmin est faux. Par exemple, <p class="base" th:class="\${isAdmin} ? 'admin' : ''"> perd aussi la classe base quand isAdmin est faux. Pour conserver une classe de base, utilisez th:classappend.

3. th:classappend

th:classappend **ajoute** une valeur à l'attribut class existant, en la séparant de la valeur actuelle par une espace. Si l'expression est vide ou null, l'attribut est laissé tel quel. C'est généralement la meilleure solution pour une classe conditionnelle, car la classe de base est conservée.

```
<!-- Template -->
<p class="base-class" th:classappend="${isAdmin} ? 'admin-class' : 'user-class'">
  Texte 1
</p>

<p th:class="${isAdmin} ? 'base-admin-class' : 'base-user-class'"
  th:classappend="${isAdmin} ? 'admin-class' : 'user-class'">
  Texte 2
</p>
```

Si isAdmin est vrai :

```
<p class="base-class admin-class">Texte 1</p>
<p class="base-admin-class admin-class">Texte 2</p>
```

Si isAdmin est faux :

```
<p class="base-class user-class">Texte 1</p>
<p class="base-user-class user-class">Texte 2</p>
```

Dans le second paragraphe, `th:class` et `th:classappend` sont présents sur la **même balise**. Cela fonctionne car Thymeleaf traite d'abord `th:class` (qui fixe la valeur de base), puis `th:classappend` (qui ajoute la suite).

Variante sans partie « sinon »

Lorsque la classe ne s'ajoute que dans un cas, on peut omettre la partie : `valeurSiFaux` : si la condition est fausse, le résultat est `null` et rien n'est ajouté.

```
<li class="menu-item" th:classappend="${item.selected} ?
'active'">Accueil</li>
```

Plusieurs conditions

Une balise ne peut porter qu'un seul attribut `th:classappend` (un même attribut ne peut pas apparaître deux fois). Pour combiner plusieurs conditions, on les regroupe dans une seule expression avec la substitution de littéraux `|...|` :

```
<button class="btn"
        th:classappend="|${primary ? 'btn-primary' : 'btn-secondary'} ${large
? 'btn-lg' : ''}|">
    Valider
</button>
```

Avec `primary = true` et `large = false`, on obtient `class="btn btn-primary "` (une espace finale sans conséquence pour le navigateur). Au-delà de deux conditions, la lisibilité chute : préparez alors la chaîne de classes dans le contrôleur ou dans un objet de présentation.

Exemple pratique : ligne d'un tableau et formulaire

```
<!-- Une ligne sur deux colorée (voir Le document sur Les boucles) -->
<tr th:each="p, state : ${people}"
    th:classappend="${state.odd} ? 'odd-row' : 'even-row'">
    ...
</tr>

<!-- Champ en erreur dans un formulaire Spring -->
<input type="text" th:field="*{email}"
        th:classappend="${#fields.hasErrors('email')} ? 'is-invalid'" />
```

4. th:style

`th:style` crée l'attribut `style` lors de la génération du HTML, ou **remplace** celui qui existe.

```
<!-- Template -->
<p style="color: blue;" th:style="${isAdmin} ? 'color: blue;' : 'color:
```

```
black;'">
  Texte 1
</p>

<p th:style="${isAdmin} ? 'color: blue;' : 'color: black;'">
  Texte 2
</p>

<p th:style="${isAdmin} ? 'color: blue;' : ''">
  Texte 3
</p>
```

Si isAdmin est vrai :

```
<p style="color: blue;">Texte 1</p>
<p style="color: blue;">Texte 2</p>
<p style="color: blue;">Texte 3</p>
```

Si isAdmin est faux :

```
<p style="color: black;">Texte 1</p>
<p style="color: black;">Texte 2</p>
<p>Texte 3</p>
```

Comme pour th:class, une valeur vide supprime l'attribut style en entier.

Cas d'usage fréquent : un style calculé à partir de données, par exemple la largeur d'une barre de progression :

```
<div class="bar" th:style="|width: ${percent}%|"></div>
```

5. th:styleappend

th:styleappend **ajoute** une valeur à la fin de l'attribut style, séparée par une espace. Si l'expression est vide ou null, l'attribut est laissé tel quel.

```
<!-- Template -->
<p style="background: #eee;"
  th:styleappend="${isAdmin} ? 'color: blue;' : 'color: black;'">
  Texte 1
</p>

<p th:style="${isAdmin} ? 'font-weight: bold;' : 'font-style: italic;'"
  th:styleappend="${isAdmin} ? 'color: blue;' : 'color: black;'">
  Texte 2
</p>
```

Si isAdmin est vrai :

```
<p style="background: #eee; color: blue;">Texte 1</p>
<p style="font-weight: bold; color: blue;">Texte 2</p>
```

Si `isAdmin` est faux :

```
<p style="background: #eee; color: black;">Texte 1</p>
<p style="font-style: italic; color: black;">Texte 2</p>
```

Attention au point-virgule. Thymeleaf ne fait que concaténer les deux valeurs avec une espace : il n'ajoute **pas** de ; entre elles. Si le style existant ne se termine pas par un point-virgule, le résultat est invalide : `style="background: #eee" + color: blue` donne `background: #eee color: blue`, que le navigateur ignore. Terminez donc toujours chaque déclaration CSS par ;, dans le HTML comme dans les expressions.

6. Bonnes pratiques

- **Préférez les classes aux styles.** Un `th:classappend` qui applique une classe CSS définie dans une feuille de style est plus lisible et plus facile à maintenir qu'un style en ligne. Réservez `th:style` aux valeurs réellement calculées (largeur, couleur issue des données).
- **Classe de base + `th:classappend`.** Ce motif évite la suppression accidentelle de la classe de base lorsque l'expression est vide.
- **Politique de sécurité du contenu (CSP).** Une politique CSP stricte peut bloquer les attributs style en ligne, ce qui est une raison supplémentaire de privilégier les classes.
- **Valeurs venant de l'utilisateur.** N'insérez jamais telle quelle dans `th:style` ou `th:class` une valeur saisie par un utilisateur : elle pourrait injecter du CSS indésirable. Choisissez la valeur parmi une liste connue (par exemple avec un `th:switch`).
- **Dans les boucles,** la variable d'état fournit `odd`, `even`, `first`, `last` pour styliser les lignes (voir le document sur les boucles).

Résumé

- `th:class` et `th:style` **remplacent** l'attribut ; si leur valeur est vide, l'attribut est supprimé en entier.
- `th:classappend` et `th:styleappend` **ajoutent** à l'attribut existant, avec une espace comme séparateur ; une valeur vide ou `null` n'ajoute rien.
- `th:class` est traité avant `th:classappend` : on peut les combiner sur la même balise.
- Terminez chaque déclaration de style par ; pour que la concaténation reste valide.
- Écrivez les valeurs entre apostrophes et privilégiez les classes CSS aux styles en ligne.

Exercices proposés

1. Afficher un tableau de produits dont la ligne est rouge si le stock est nul, orange s'il est inférieur à 5 et verte sinon, avec `th:classappend` (conditionnels imbriqués).

2. Expliquer pourquoi `<p class="base" th:class="{x} ? 'a' : ''">` ne produit pas le même HTML que `<p class="base" th:classappend="{x} ? 'a' ">` quand `x` est faux.
3. Créer une barre de progression dont la largeur dépend d'un pourcentage du modèle, avec `th:style` et la substitution de littéraux.
4. Corriger : `<p style="color: red" th:styleappend="'font-weight: bold'">` produit un style invalide. Pourquoi ?
5. Ajouter la classe `is-invalid` à tous les champs d'un formulaire qui ont une erreur de validation, avec `#fields.hasErrors`.